Programming Techniques 2026–2027

Course exercise 1: direct N-body simulation with OpenMP

Hannu Parviainen

Universidad de la Laguna

October 19, 2026

Exercise 1: direct N-body simulation

Simulate N particles that interact through gravity, and parallelise the simulation with OpenMP.

The exercise has three parts:

1. modules for the geometry and the particles,
2. a program that reads its setup from a file, writes the results into a file, and is built with a makefile,
3. OpenMP parallelisation and a timing study.

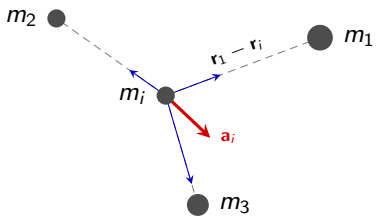
Starting code in `lectures/exercise_1`:

- `leapfrog.f90`: the simulation, with plain arrays. It reads standard input: `./leapfrog < input.dat`.
- `input.dat`: three bodies on a figure-eight orbit.
- `random_sphere.f90`: creates N particles in a sphere.

The input file format:

```
dt                  ! time step
dt_out              ! time between outputs
t_end               ! end time
n                   ! number of particles
m x y z vx vy vz    ! one line per particle
```

Gravity between N particles



- Every particle pulls every other particle towards itself (blue), with a strength $\propto m_j/r^2$.
- The acceleration of particle i is the vector sum of all the pulls (red):

$$\mathbf{a}_i = \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}$$

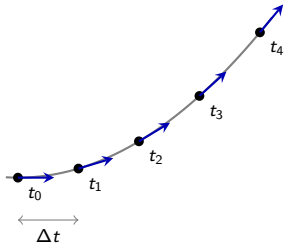
- The units are chosen so that $G = 1$.
- N particles have $N(N-1)/2$ pairs: 3 for 3 particles, but about 500 000 for 1000 particles. This is where the time goes.

From forces to motion

- The position $\mathbf{r}_i$ and the velocity $\mathbf{v}_i$ of each particle change as

$$\frac{d\mathbf{r}_i}{dt} = \mathbf{v}_i, \qquad \frac{d\mathbf{v}_i}{dt} = \mathbf{a}_i(\mathbf{r}_1, \ldots, \mathbf{r}_N).$$

- The acceleration of each particle depends on the positions of all the others, so the equations are coupled. For $N \geq 3$, they have no general exact solution.
- Instead, we advance the system in small time steps Δt, short compared with its fastest changes, such as orbital periods and close encounters.
- In a long simulation, small errors in each step can add up. A good method keeps them from growing.

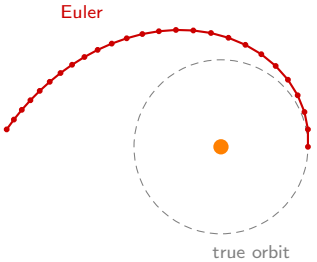


A trajectory sampled at the times $t_n = n\,\Delta t$. The velocity (blue) changes within each step, and the method must approximate this change.

The simplest method: Euler

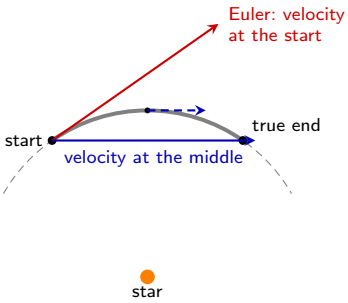
$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t$$
$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t$$

- $\mathbf{r}_n$ is the position at time $t_n = n\Delta t$.
- Euler's method uses the velocity and the acceleration at the *start* of the step for the whole step.
- Simple, but systematically wrong: on a circular orbit, the planet spirals outwards and finally escapes. A smaller Δt only slows this down.



A circular orbit computed with Euler's method, with about 31 steps per orbit. After 25 steps, the planet is already 2.5 times further from the star.

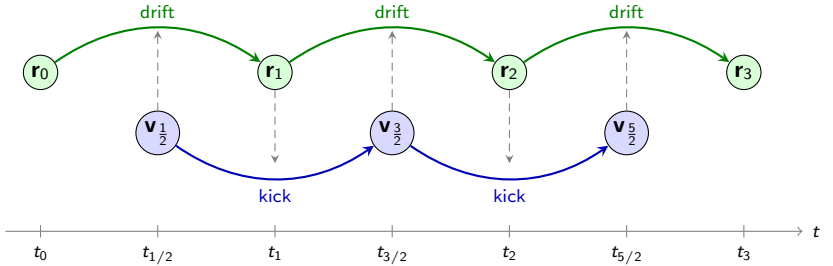
Why Euler goes wrong



- On an orbit, the velocity turns all the time. The velocity at the start points away from the orbit, so every Euler step lands outside it.
- The velocity at the *middle* of the step lands (almost) at the right place.

Lesson: use the value at the *middle* of the step.

Leapfrog: positions and velocities take turns



- Positions live at whole steps, velocities at half steps. They jump over each other, like children playing leapfrog.
- The dashed arrows show that each jump uses the other quantity from exactly the middle of the jump.

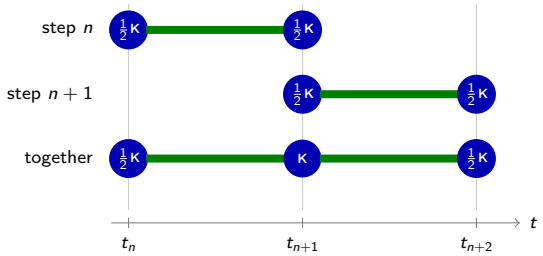
$$\text{drift:} \quad \mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1/2}\,\Delta t$$

$$\text{kick:} \quad \mathbf{v}_{n+3/2} = \mathbf{v}_{n+1/2} + \mathbf{a}_{n+1}\,\Delta t$$

$\mathbf{a}_{n+1}$ is computed from the positions $\mathbf{r}_{n+1}$.

Kick-drift-kick: back to whole steps

- **Kick:** change the velocities. Gravity gives the particles a push.
- **Drift:** change the positions. The particles coast with their current velocities.
- Velocities at half steps are awkward: the input gives $\mathbf{v}_0$, not $\mathbf{v}_{1/2}$, and the output needs $\mathbf{r}$ and $\mathbf{v}$ at the same times.
- Fix: split each kick into two half kicks. Two half kicks in a row are one full kick, so the result is the same leapfrog.



Blue circles: kicks, which happen at one instant. Green bars: drifts, which last a whole step. In the middle, the half kick that ends step n and the half kick that starts step $n+1$ add up to one full kick.

One leapfrog step in detail

At the start of step n, we know $\mathbf{r}_n$, $\mathbf{v}_n$, and $\mathbf{a}_n$ (from the previous step).

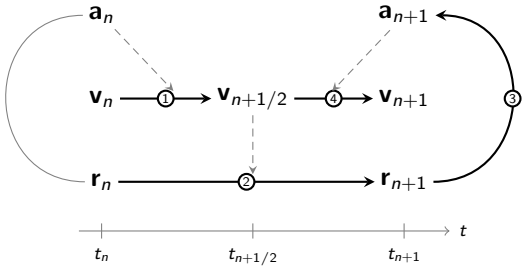
① kick $\mathbf{v}_{n+1/2} = \mathbf{v}_n + \mathbf{a}_n \, \Delta t/2$

② drift $\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1/2} \, \Delta t$

③ force $\mathbf{a}_{n+1}$ from the positions $\mathbf{r}_{n+1}$

④ kick $\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \mathbf{a}_{n+1} \, \Delta t/2$

- $\mathbf{a}_{n+1}$ is reused as $\mathbf{a}_n$ in the next step, so each step computes the forces only once, like Euler.
- Step 3 takes almost all the time, so it is the step to parallelise.



Solid arrows: the four steps. Dashed arrows: what each step uses.

Leapfrog in code

```
! Accelerations at t = 0
call accelerations(m, r, a)

do step = 1, n_steps
  v = v + a * dt / 2           ! kick
  r = r + v * dt               ! drift
  call accelerations(m, r, a)  ! force
  v = v + a * dt / 2           ! kick
end do
```

This is the loop in `leapfrog.f90`, where the force calculation is written out in full instead of being a subroutine.

Choosing Δt

- Δt must be short compared with the fastest changes in the system, such as close encounters between particles.
- Test: run with Δt and with $\Delta t/2$. If the results differ too much, Δt is too large.

Testing the code

In `input.dat`, three equal masses chase each other along a figure eight, and return to their starting positions every 6.33 time units. A correct program reproduces this orbit again and again.

Exercise 1: modules

1. Create a module `geometry` in the file `geometry.f90` that contains:
   - the types `vector3d` and `point3d`, both with 64-bit real components `x`, `y`, and `z`,
   - the functions `sumvp`, `sumpv`, `subvp`, `subpv`, `mulrv`, `mulvr`, and `divvr` for adding and subtracting points and vectors, and for multiplying and dividing vectors by reals,
   - operators that match these functions,
   - a function `distance` that returns the distance between two points.
2. Create a module `particle` in the file `particle.f90` that uses the `geometry` module and contains a type `particle3d` with the components:
   - `p`, a `point3d` for the position,
   - `v`, a `vector3d` for the velocity,
   - `m`, a real for the mass.

Why two types with the same components?

A point is a location (a position). A vector is a displacement (a velocity, an acceleration, or the difference of two points). Point + vector is a point and point − point is a vector, but point + point means nothing. With separate types, the compiler rejects such mistakes.

Exercise 1: the program

3. Change the `leapfrog.f90` code to use the `geometry` and `particle` modules, and store the program in the file `ex1.f90`.
4. Read the simulation setup from a file whose name is given as a command-line argument.
5. Write the simulation data into the file `output.dat`. Each line contains the state of the simulation at one time as `t p1x p1y p1z p2x p2y p2z ... pnx pny pnz`.
6. Write a makefile for the project.

Check your program with `input.dat`: the three bodies should follow the same figure-eight orbit again and again.

Exercise 1: OpenMP

7. Parallelise the calculation of the accelerations with OpenMP.
   - The program must compile and give the same results both with and without `-fopenmp`. Add both builds to the makefile.
   - Hint: the loop over the pairs of particles updates the accelerations of both particles of the pair. What happens if two threads update the same particle at the same time?
8. Measure how the run time depends on the number of particles and the number of threads.
   - Create the initial conditions with `random_sphere.f90` for $N = 1000$, 2000, 5000, and 10 000. Give it a seed, so that every run uses the same particles.
   - Run each case with 1, 2, 4, and 8 threads (set `OMP_NUM_THREADS`).
   - Set `dt_out` equal to `t_end` while timing, so that writing the output does not dominate.
   - Report the run times and the speed-ups in a table or a figure in the `README.md`, and explain what you see.

Exercise 1: submission

9. Write a `README.md` for the exercise that explains how the code works, how to compile and run it, and the results of the timing study.
10. Create a pull request (PR) with your code, and add all the course participants as reviewers.
11. Review the code in the other students' PRs, and write at least one comment on each.
12. (Optional) Revise your code based on the comments you have received.
13. In the PR, tag the teacher and say that you have finished the exercise.

Exercise 1: scoring

Scoring is based on

- Code (80%)
  1. Functionality: the code works as described, also without OpenMP.
  2. Understandability: the code is clear to read and logical.
  3. Comments: all the functionality in the modules and the program is documented with comments.
  4. Use of guidelines: the code uses the type, function, and file names exactly as written in the exercise.
  5. Timing study: the measurements are complete and explained.
- Collaboration (20%): GitHub PRs, PR comments, and participation.

Bonus points for anything that goes beyond what has been taught in class, frequent commits, and interaction (asking and answering questions, and so on). If in doubt, ask!

Notes: Save all the source files in `students/<initials>/course_exercise_1`. All the real variables must be 64-bit. The code should compile without warnings with `gfortran -Wall`. Commit your changes often, and feel free to work with branches.